Text Vectorization II: Word Embeddings and other Embeddings representations

Natural Language Processing
Master in Applied Artificial Intelligence

Lorena Calvo Bartolomé
lcalvo@pa.uc3m.es

November 17, 2023

Department of Signal Theory and Communications
Universidad Carlos III de Madrid

Table of Contents

Problems with words as discrete symbols

- The **BoW** model is very limited, it represents the frequencies of each word in a piece of text and nothing else
- **TF-IDF** is based on the BoW model, therefore it does not capture position in texts, semantics, co-occurrences in different documents, etc.
- **One-hot encoded vectors** also presents severe limitations
  - ➔ Vectors become more sparse as the vocabulary grows
  - ➔ Every word is orthogonal to each other $\Rightarrow$ **They do not capture meaning!**

But how do we know what meaning is?

Distributional semantics

Distributional hypothesis

Words which frequently appear in similar contexts have similar meaning

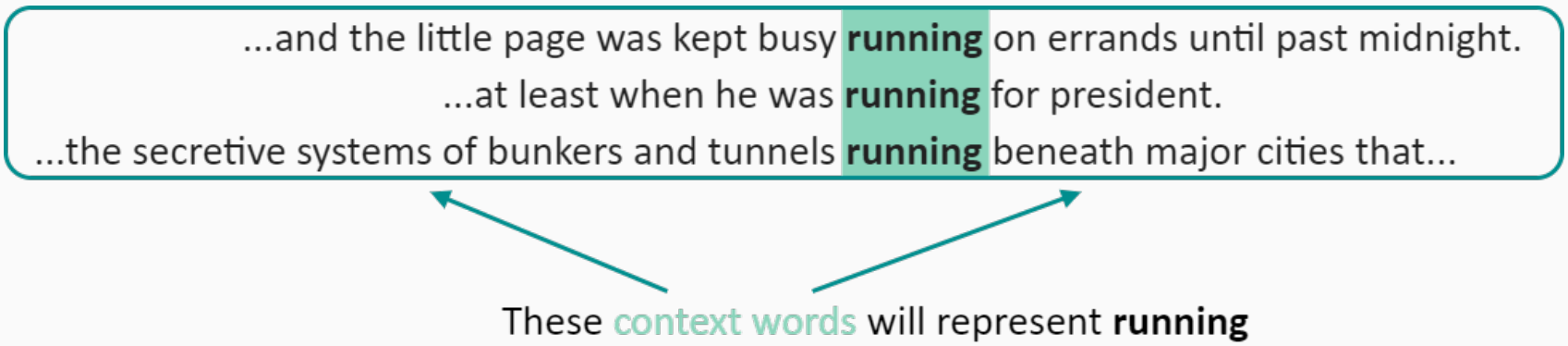
» *"You shall know a word by the company it keeps"* (J. R. Firth, 1957)

Main idea

"To capture meaning" ⇔ "to capture context"

» We need to encode information about word contexts into the word representation

The **context of a word** appearing in a text is the set of nearby words

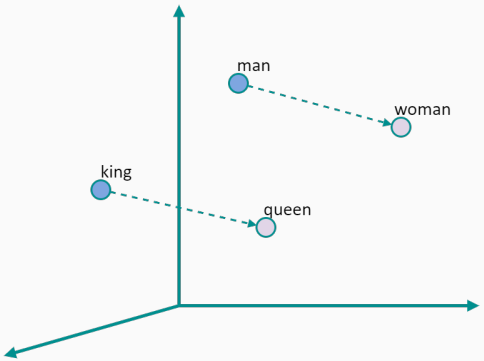


Word meaning as a vector

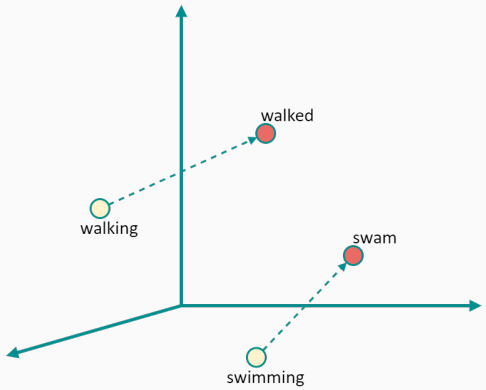
Word embeddings

Type of word representation that allows words with similar meaning to have close representations

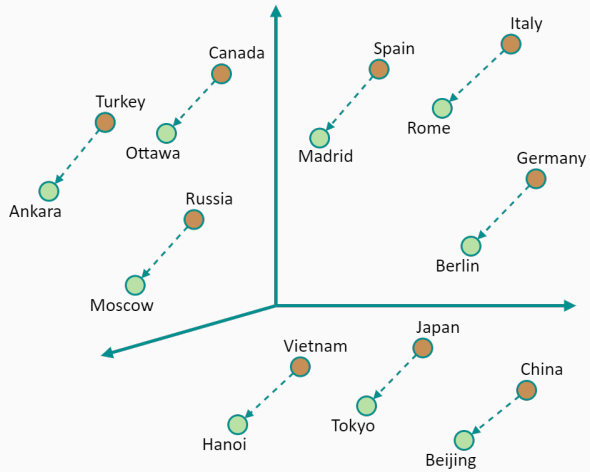
- For each word, we will have a dense vector chosen so that it is similar to other word vectors that appear in a similar context
  - » The similarity is measured as the vector **dot** (scalar) **product**



Male-Female



Verb Tense



Country-Capital

- Embedding Projector

Methods to Obtain Word Embeddings

Main idea

"To capture meaning" $\Leftrightarrow$ "to capture context"

» We need to encode information about word context into the word representation

How do we put information about contexts into word vectors?

- **Count-based methods:** Co-occurrence counts, PPMI, LSA...
- **Prediction-based methods:** Word2Vec, vLBL...
- **Combination of the latter:** Glove

Table of Contents

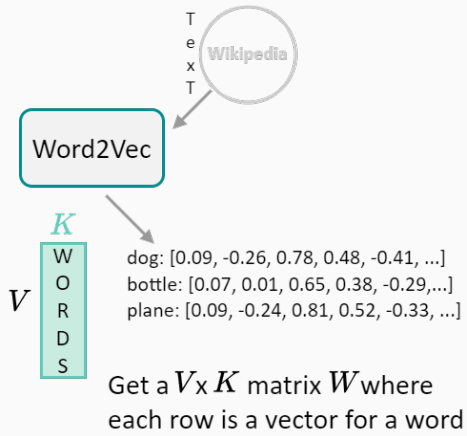
Word2Vec is a prediction-based method for learning word vectors

Motivation

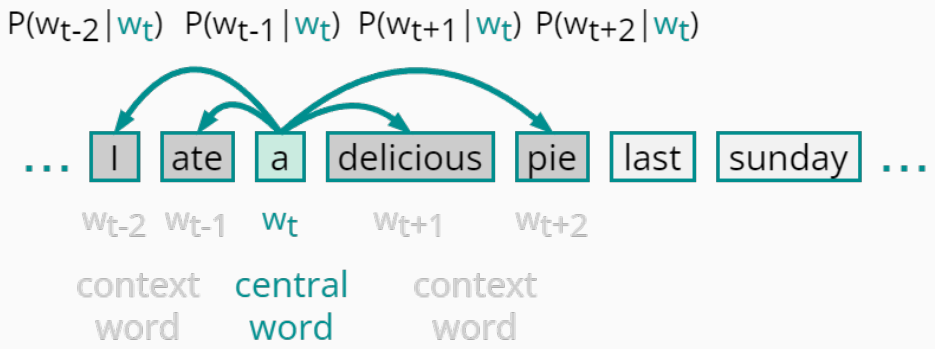
Design a model to iteratively learn to encode the probability of a word given its context, i.e., a model whose parameters are word vectors

Idea:

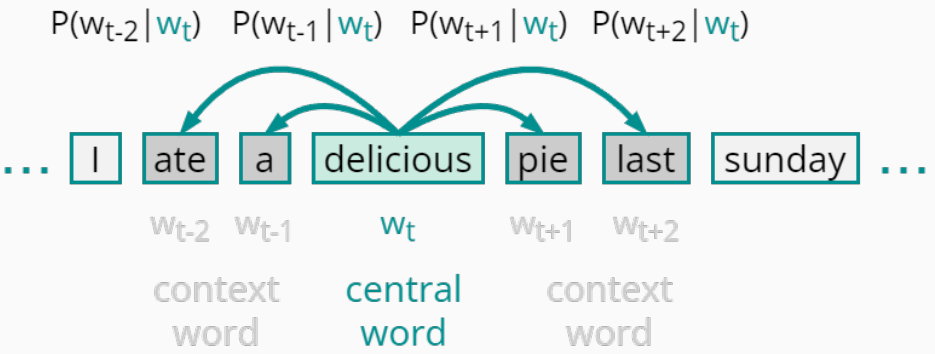
- We have a huge text corpus
- Go through the text with a **sliding window**, moving **one word at a time**. At each step, there is **one center word** and **several context words**
- Compute probabilities of context words given the central word
- Adjust the word vectors to maximize these probabilities



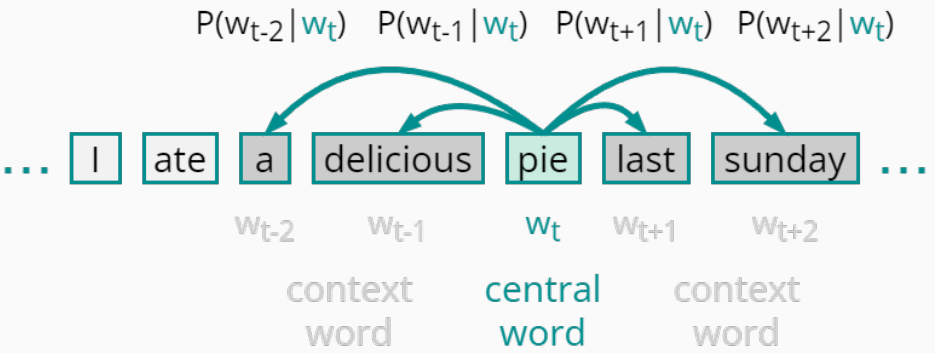
Word2Vec: Example of computing $P(w_{t+j} \mid w_t)$ in a context window of size 2



Training samples:
(a,I)
(a,ate)
(a, delicious)
(a,pie)



Training samples:
(delicious,ate)
(delicious,a)
(delicious, pie)
(delicious,last)



Training samples:
(pie,a)
(pie,delicious)
(pie, last)
(pie,sunday)

Word2Vec: Objective function

For each position $t = 1, \ldots, T$, predict context words within an **m-sized** window given the central word w_t, θ representing all variables to optimize

$$\textit{Likelihood} = L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta)$$

all variables to be optimized

The **objective function** $J(\theta)$ is the average negative log-likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \underbrace{\sum_{t=1}^{T}}_{\text{go over the text}} \underbrace{\sum_{\substack{-m \le j \le m \\ j \neq 0}}}_{\text{with a sliding window}} \log P(w_{t+j} \mid w_t; \boldsymbol{\theta})$$

Negative Log Likelihood

Minimizing objective function $\Leftrightarrow$ Maximizing predictive accuracy

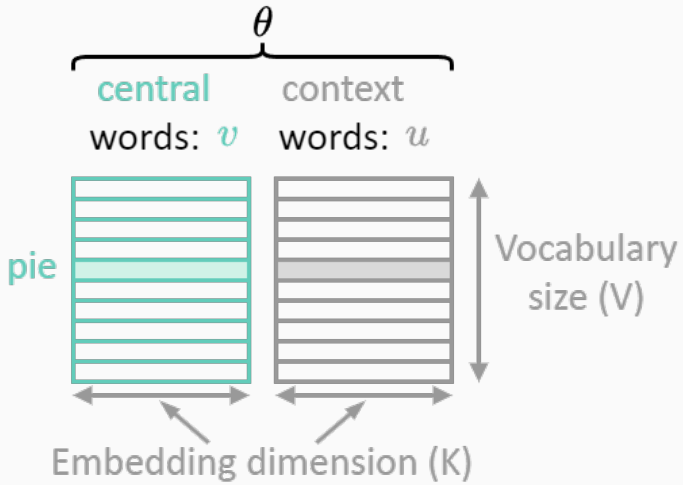
Word2Vec: Objective function

Goal: Minimize the objective function $J(\theta)$

How do we calculate $P(w_{t+j} \mid w_t, \theta)$?

For each word w, we define two vectors:

- v_w when w is a center word (c)
- u_w when w is a context word (o)



Exponentiation makes anything positive

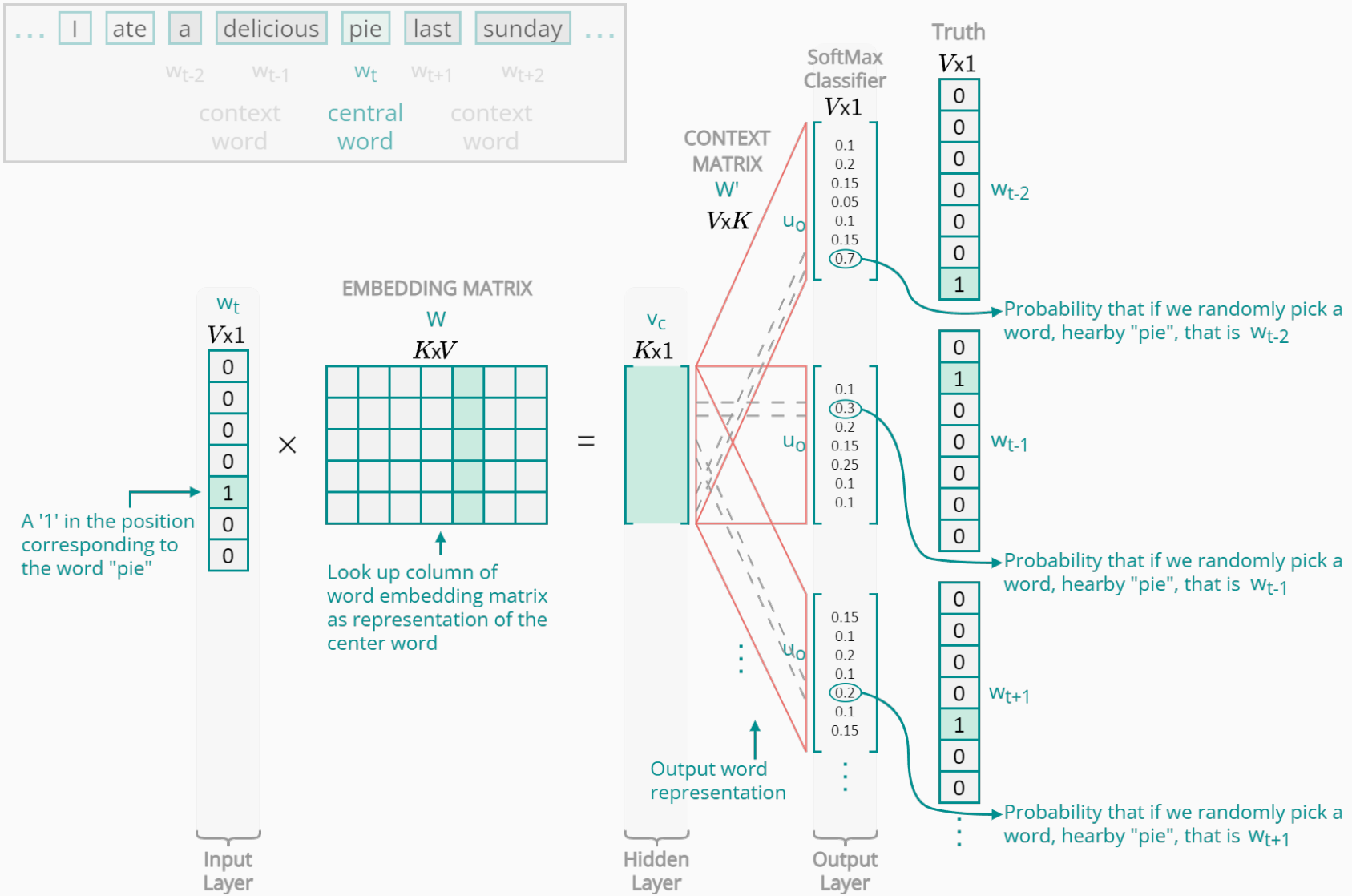
Dot product measures the similarity $\{o, c\} : \boldsymbol{u}^T\boldsymbol{v} = \boldsymbol{u} \cdot \boldsymbol{v} = \sum_{i=1}^{n} u_i v_i$

$$P(o \mid c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in \mathcal{V}} \exp(u_w^T v_c)}$$

Softmax function

Normalize over the entire vocabulary to give probability distribution

Word2Vec: Training the model



- Wevi: word embedding visual inspector

Word2Vec: Optimization via Gradient Descent

Idea

Gradually **adjust the parameters** $\theta \in \mathcal{R}^{2KV}$ **to minimize the loss** $J(\theta)$

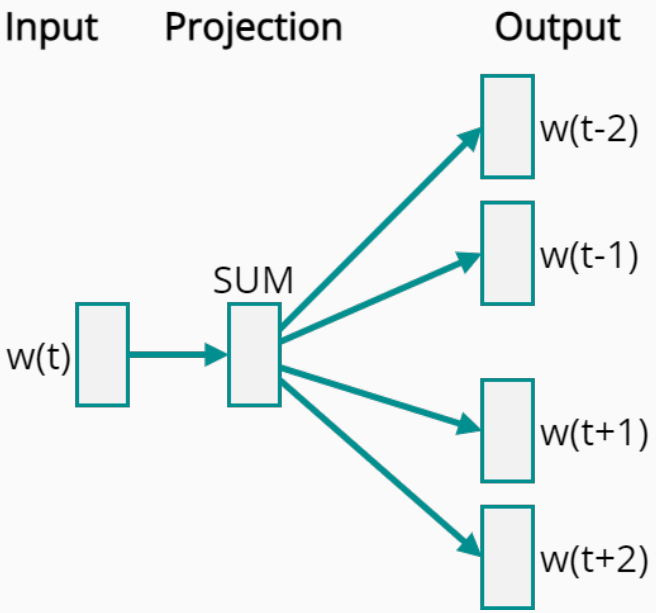
- **Update equation:**

$$\theta^{\text{new}} = \theta^{\text{old}} - \underbrace{\alpha}_{\text{Learning rate or step size}} \nabla_\theta J(\theta)$$

- Perform an update for **one word at a time**:
  - ➔ Each update is for a single pair of $\{o, c\}$
- **Problem:** $J(\theta)$ is a function of all windows in the corpus, the computational cost of calculating $\nabla_\theta J(\theta)$ is too high
- **Solution:** **Stochastic GD**
  - ➔ Repeatedly sample windows and update after each one

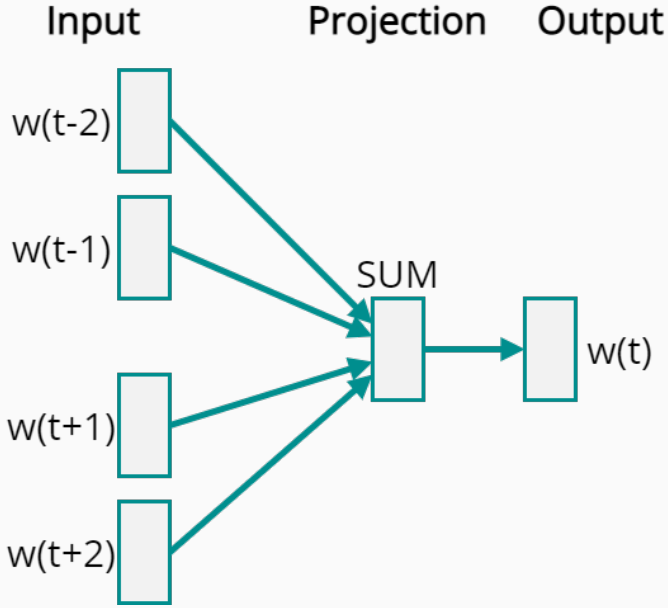
$$\boldsymbol{\theta} = \begin{bmatrix} v_a \\ v_{ate} \\ \vdots \\ v_{sunday} \\ u_a \\ u_{ate} \\ \vdots \\ u_{sunday} \end{bmatrix} \in \mathcal{R}^{2KV}$$

Word2Vec variants: Skip-Gram and CBOW



Skip-gram

- Predict surrounding context words given a central word
- Needs less data and represents better “weird” words



CBOW

- Predict a center word from the surrounding context
- Faster and better representation for frequent words

Effective training of Word2Vec: Negative sampling

Problem: Loss functions for CBOW and Skip-Gram models are expensive to compute because of the softmax normalization

➔ $O(|V|)$ time to update or evaluate the objective function!

Solution:

- We evaluate the softmax for only **a few negative samples**, i.e., a set of K randomly selected words
- Milokov's paper says that selecting 5-20 words works well for smaller datasets, and you can get away with only 2-5 words for large datasets
- On average over all updates we will update each vector a sufficient number of times, and the vectors will still be able to learn the relationships between words quite well
- The probability for selecting a word as a negative sample is related to its frequency

Summary of Word2Vec

- Go through each word of the whole corpus
- Predict the surrounding words of each word
- This captures the co-occurrence of words one at a time
  - ➔ **This is not really efficient...**

Why not capture co-occurrence counts directly?

Count-based methods

Motivation

Information about word context is encoded into word vectors based on global corpus statistics into a co-occurrence matrix X

2 options:

- **Window:** Similar to Word2Vec, uses a window around each word, capturing both syntactic (POS, e.g., verbs are gonna be closer together than they are to nouns) and semantic information
- **Word-document co-occurrence matrix:** Captures general topics (e.g., all sport-related terms will have similar entries)

Need to define:

- **What is context** (e.g. surrounding words in an L-sized window)
- **How to compute matrix elements,** i.e. notion of association (e.g., co-occurrence counts, PPMI, etc.)

Example: Window based co-occurrence matrix

- Window length 1 (more common: 5-10)
- Symmetric (irrelevant whether left or right context)
- Example corpus:
  - I like deep learning.
  - I like NLP.
  - I enjoy flying.

counts	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying0	0	1	0	0	0	0	0	1
.	0	0	0	0	1	1	1	0

Problems with simple co-occurrence vectors

Problems:

- Increase in size with vocabulary
- Very high dimensional: Require a lot of storage
- Subsequent classification models have sparsity issues
  - ➔ Models are less robust

Solution: Reduce the dimensionality of X via Singular Value Decomposition

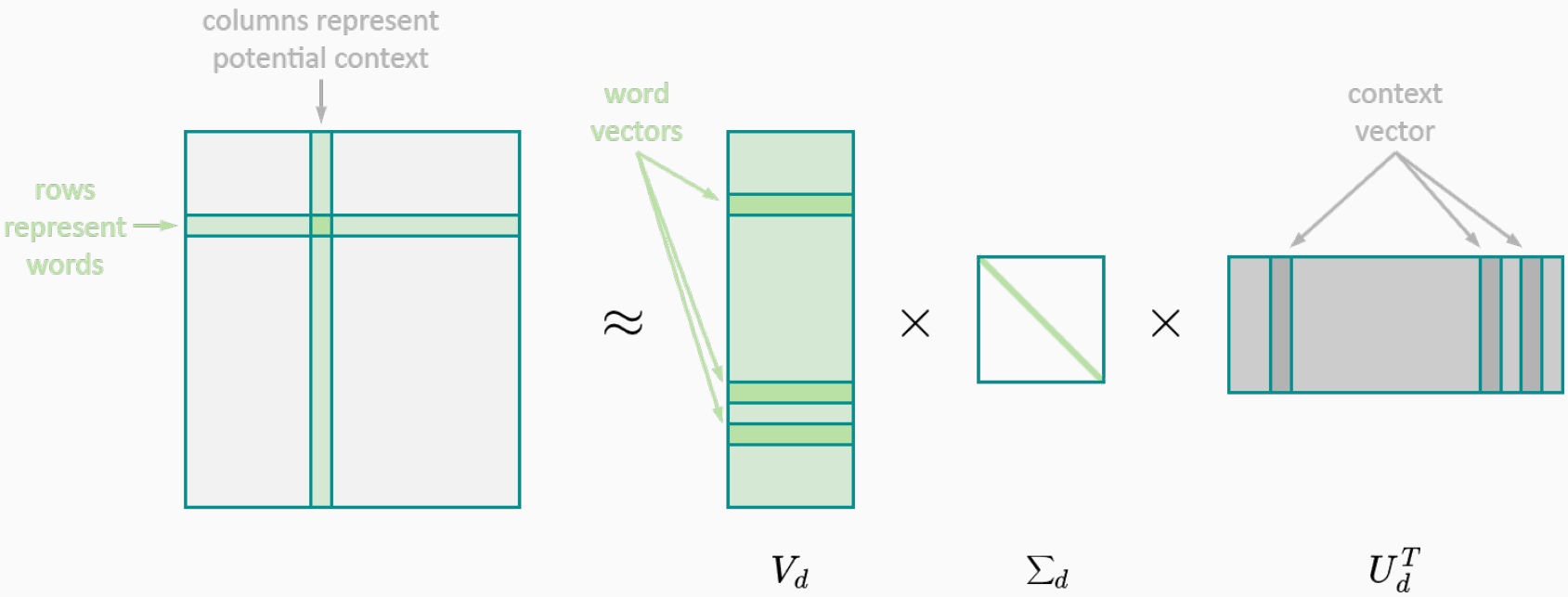
Idea

Store “most” of the important information in a fixed, small number of dimensions, i.e., a dense vector

Improvements:

- Ignore function words (the, a, etc.)
- Ramped windows that count closer words more
- ...

Summary on Count-based methods



1. Construct a word-context matrix

2. Reduce dimensionality by applying SVD (Singular Value Decomposition)

Problems with SVD

- Computational cost scales quadratically for $n \times m$ matrix: $O(mn^2)$ flops (when $n < m$)
  - Bad for millions of words or documents
- Hard to incorporate new words or documents
- Different learning regime than other DL models

Global Vectors for Word Representation (GloVe)

Count-Based vs Prediction-Based

- Fast training
- Efficient usage of global statistical information
- Primarily used to capture word similarities
- Disproportionate importance given to large counts

- Scales with corpus size
- Inefficient usage of statistics
- Generate improved performance on other tasks
- Can capture complex patterns beyond word similarity

GloVe

Combination of count and prediction-based methods that makes use of **global information** for learning word vectors via SGD

GloVe: Intuition

Motivation

Word co-occurrence is the most important statistical information available for the model to "learn" the word representation

Probability and Ratio	$w_k =$ solid	$w_k =$ gas	$w_k =$ water	$w_k =$ fashion
$P(w_k \mid ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(w_k \mid steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(w_k \mid ice)/P(w_k \mid steam)$	8.9	8.5×10^{-2}	1.36	0.96

- The word `ice` is more similar to `solid` than `steam`, so $P(solid \mid ice) > P(solid \mid steam)$
- Neither `ice` nor `steam` are related to `fashion`, yet if we consider the standalone probability values for $P(fashion \mid ice)$ or $P(fashion \mid steam)$, it will not help us to infer the lack of relationship

Idea: Consider co-occurrence probabilities as a ratio and use it as a word representation, since vs raw probabilities this ratio is better able to:

- distinguish relevant words from irrelevant words
- discriminate between relevant words

GloVe: Loss function

Let $w \in \mathcal{R}^d$ be word vectors and $\tilde{w} \in \mathcal{R}^d$ separate context word vectors:

$$F(w_i, w_j, \tilde{w}_k) \approx \frac{p_{ik}}{p_{jk}}.$$

$$F(w_i, w_j, \tilde{w}_k) = \frac{\exp\left(w_i^T \tilde{w}_k\right)}{\exp\left(w_j^T \tilde{w}_k\right)} \approx \frac{p_{ik}}{p_{jk}}$$

$$F(w_i, w_j, \tilde{w}_k) = \frac{\exp\left(w_i^T \tilde{w}_k\right)}{\exp\left(w_j^T \tilde{w}_k\right)} \approx \frac{p_{ik}}{p_{jk}}. \rightarrow \exp\left(w_i^T \tilde{w}_k\right) \approx \alpha p_{ik} \qquad p_{ik} = X_{ik}/X_i$$

$$w_i^T \tilde{w}_k \approx \log \alpha + \log\left(X_{ik}\right) - \log\left(X_i\right)$$

$$w_i^T \tilde{w}_k + b_i + \tilde{b}_k \approx \log\left(X_{ik}\right)$$

Center word bias

Context word bias

GloVe: Loss function

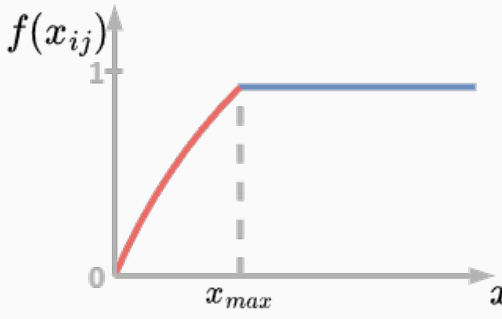
$$J(\theta) = \sum_{i,j=1}^{V} f(X_{ij})\,(w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

Weighting function to:

- penalize rare events
- not to over-weight frequent events

Parameters:

- different vectors for central and context words
- scalar bias term for each word vector



$$\begin{cases} (x/x_{max})^a & \text{if } x < x_{max}, \\ 1 & \text{otherwise.} \end{cases}$$

$$a = 0.75, x_{max} = 100$$

- Fast training
- Scalable to huge corpora
- Good performance even with small corpus and small vectors

GloVe: Training the model

1. Create a $V \times V$ weighted, distance-based co-occurrence matrix, X, with a context window C, where each entry X_{ij} corresponds to each word i occurring within a distance of word j:
   - $X_{ij} = 1$, if i, j co-occur with no words in between
   - $X_{ij} = 1/2$, if i, j co-occur with one word in between
   - etc., until the maximum C is reached

context window

the dog eats a carrot rather than the treats.

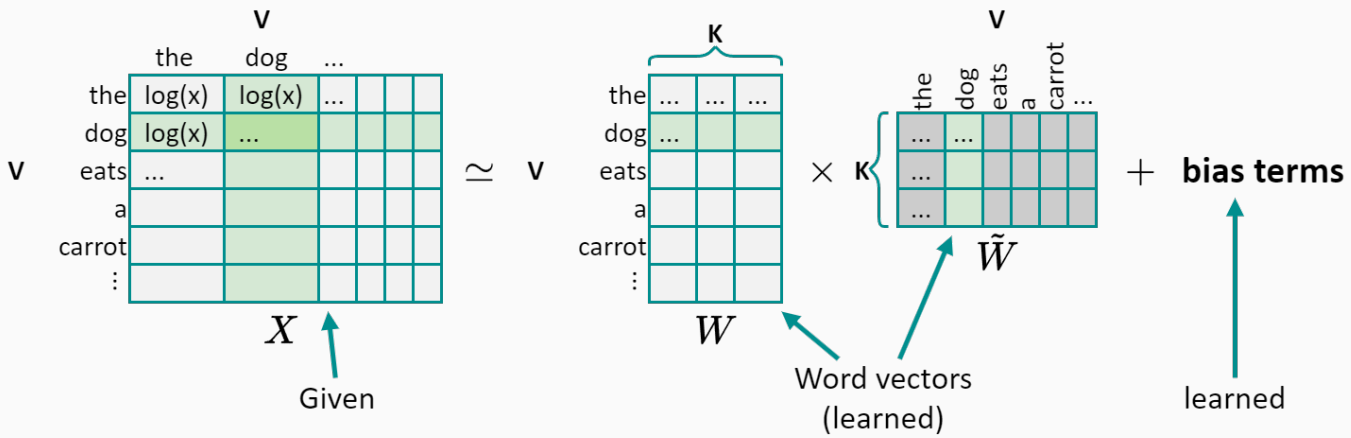
i	j	x_{ij}
the	dog	1
the	eats	1/2
the	a	1/3
dog	carrot	1

2. X becomes GloVe's target matrix. During training, the loss function J is minimized via gradient descent to predict the values of this co-occurrence matrix

GloVe: Training the model

4. We end up with W and $\tilde{W}$ from all the vectors w and $\tilde{w}$ (in columns)
5. Both capture similar co-occurrence information. It turns out, the best solution is to simply sum them up:

$$X_{final} = W + \tilde{W}$$



Trained matrices $\Leftrightarrow$ Embeddings for each word

Summary: Word2Vec vs GloVe

Word2Vec

- Tries to capture co-occurrences one window at a time
- Takes a large corpus as training data for a neural network

GloVe

- Tries to capture the counts of the overall statistics of how often these words appear together
- Uses a weighted least squares method that trains on global co-occurrence counts

- In the end, the quality of their resulting word vectors tends to be roughly similar
- Different downstream applications may prefer to either lean more toward the Word2Vec or the GloVe representation

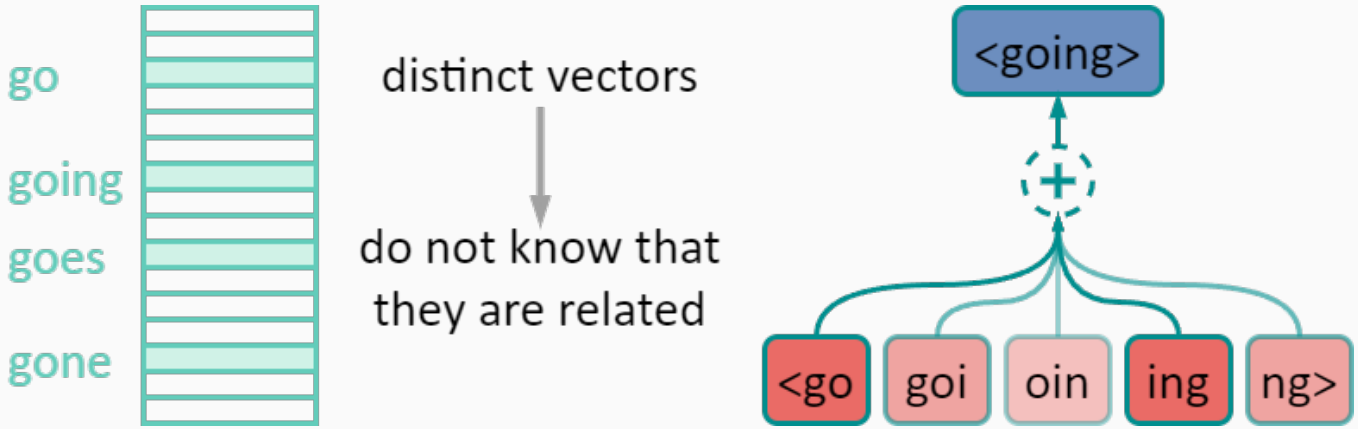
Table of Contents

Subword Embeddings

Word-level embedding models assign a distinct vector to each word, and all the information they have is what they learned from contexts

- often ignore the morphology information and represent different inflected forms of the same word by different vectors
  - ➔ helps, helped, helping ⇔ help
- cannot assign meaningful vectors to out-of-vocabulary (OOV) words
  - ➔ it may have learnt the words `dragon` + `fly`, but not `dragonfly`

Why not consider subword units?

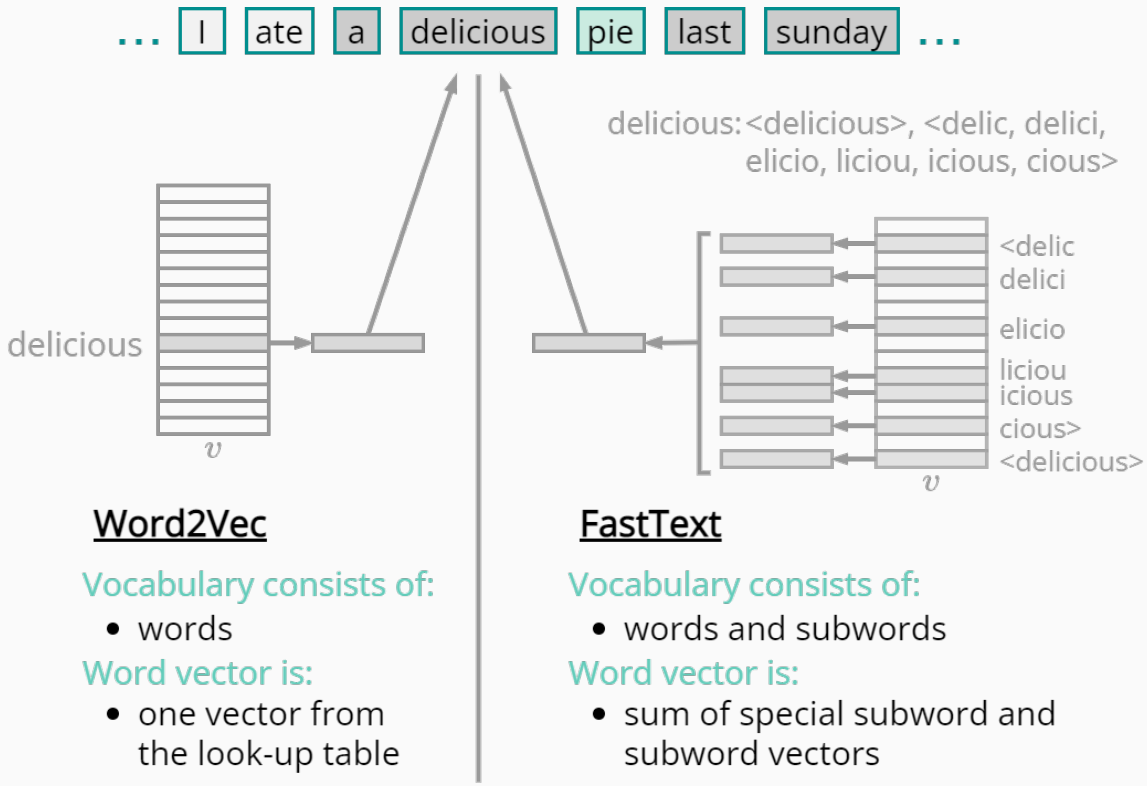


FastText is a subword embedding approach for the learning of word vectors

Motivation

To benefit from the morphological information of the words

Idea: Same as the skip-gram model but at the **subword-level**: each center word is represented by the sum of its subword vectors



FastText: Word vectors generation

For each center word w in the corpus:

1. Add special characters **"<"** and **">"** at the beginning and end of the word to distinguish prefixes and suffixes from other subwords
2. Extract groups of characters of length $\in [3, 6]$ from w
   - ➔ E.g., if $w = \textit{delicious}$ and $n = 6$, we obtain all **subwords** of length 6:
     "<delic", "delici", "elicio", "liciou", "icious", "cious>"
     and the **special subword** "<delicious>"
3. $\mathcal{G}_w$ is the union of all its subwords and its special subword
4. Letting z_g be the vector of subword g in the dictionary, the vector v_w for word w as a center word is the sum of its subword vectors:

$$v_w = \sum_{g \in \mathcal{G}_w} z_g$$

» This changes only the way we form word vectors; the whole training pipeline is the same as in Word2Vec

Summary on FastText

- **Pros:** Rare words and OOV words may obtain better representations
  - ➔ To get the embedding of an OOV word, we just need to compute the sum of the embeddings of the corresponding subwords
- **Cons:**
  - ➔ Larger vocabulary $\Rightarrow$ more model parameters
  - ➔ Higher computational complexity: to calculate the representation of a word all its subword vectors have to be summed up
  - ➔ All the extracted subwords have to be of the specified lengths, thus the vocabulary size cannot be predefined
  - ➔ Not all n-grams are morphemes
- FastText substantially improves Word2Vec when the dataset has a medium/low size, but the differences are substantially reduced for very large datasets, with Word2Vec being much more efficient in its training

Byte Pair Encoding

BPE is a compression algorithm to extract subwords

Motivation

To allow for variable-length subwords in a fixed-size vocabulary

Idea:

- Perform statistical analysis of the training dataset to discover common symbols within a word, such as consecutive characters of arbitrary length
- Starting from symbols of length 1, byte pair encoding iteratively merges the most frequent pair of consecutive symbols to produce new longer symbols
- For efficiency, pairs crossing word boundaries are not considered
- In the end, we can use such symbols as subwords to segment words

Reflections on word representations

- Up until now, we have said that we have **one representation of words**: Word2Vec, GloVe, fastText...
- **Problems:**
  1. Always assign the same vector to the same word regardless of its context
     - ➔ the word “bank” in contexts “my bank called today” and “the bank is free” has completely different meanings
  2. We just have one representation for a word, but words have different aspects: semantics, syntactic behavior, and register/connotations
     - ➔ “arrive” and “arrival”, their semantics are almost the same, but they are $\neq$ POS
     - ➔ “bathroom”, “toilet” and “WC” mean semantically the same
- **Solution:** Development of **context-sensitive** word representations

Language Modeling

Language Modeling is the task of predicting what word comes next, e.g.:

The students opened their _____ {books, laptops, exams, minds}

Language models

Traditionally defined as models capable of predicting the next word in a sentence:

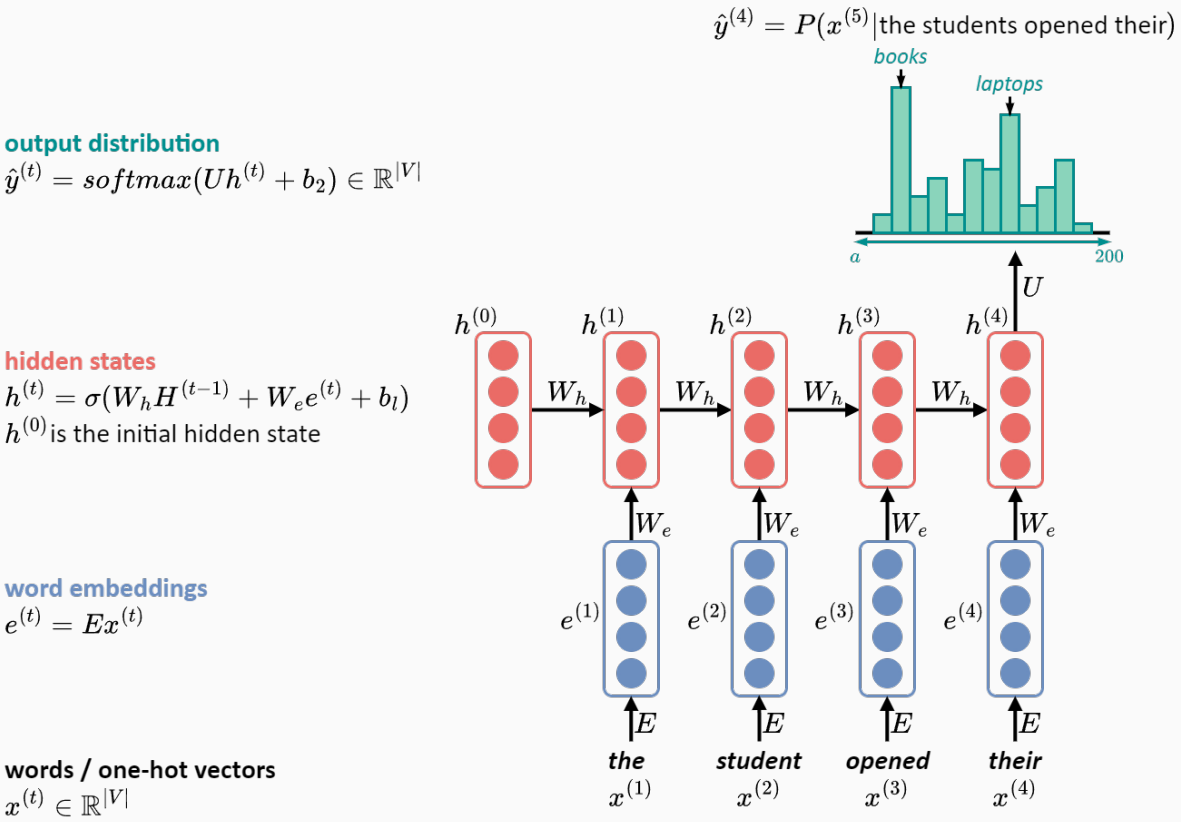
$$P_\theta(word_N \mid word_{N-1}, word_{N-2}, \ldots, word_0)$$

More generally:

$$P_\theta(text \mid context)$$

Can language models be used to obtain contextualized word vectors?

Neural Language Models for context sensitive word representations



- In an NLM, we immediately stuck word vectors through RNN layers
- Those layers are trained to predict the next word
- But those language models are producing context-specific word representations at each position!

Context-sensitive word representations

Contextual Word Embeddings

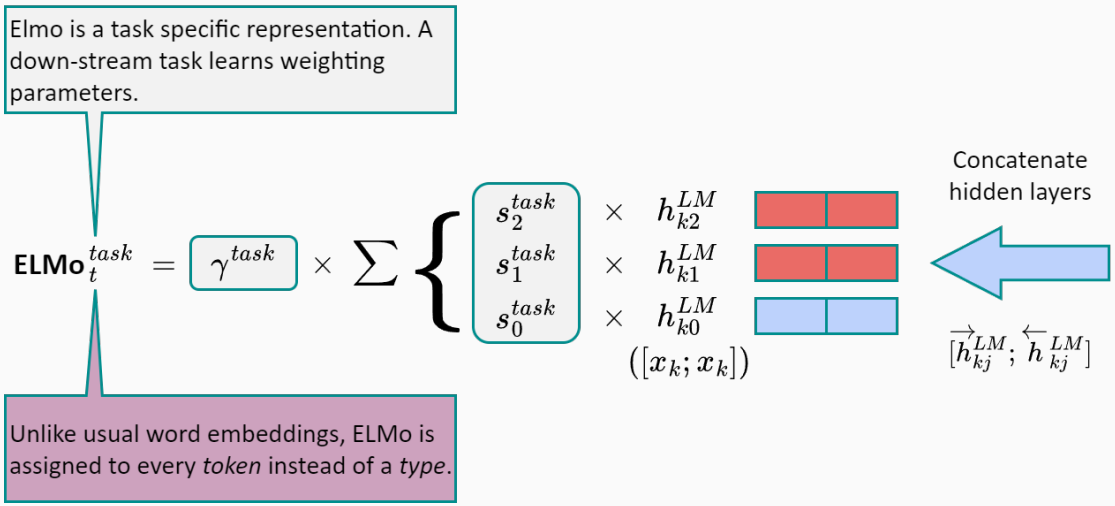
Representations of words that depend on their context, i.e., a context-sensitive representation of token x is a function $f(x, c(x))$ depending on both x and its context $c(x)$

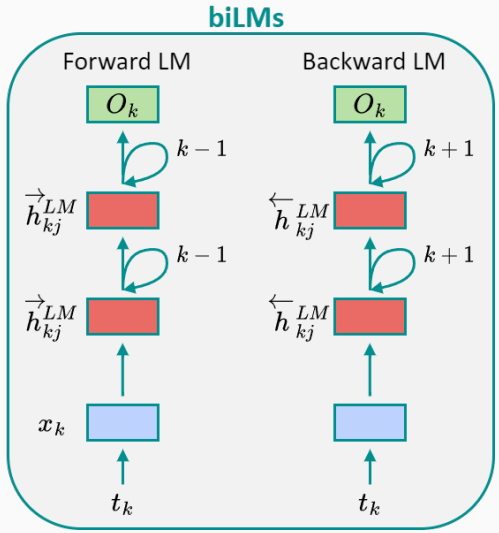
Popular context-sensitive representations include:

- **TagLM** (language-model-augmented sequence tagger)
- **CoVe** (Context Vectors)
- **ELMo** (Embeddings from Language Models)

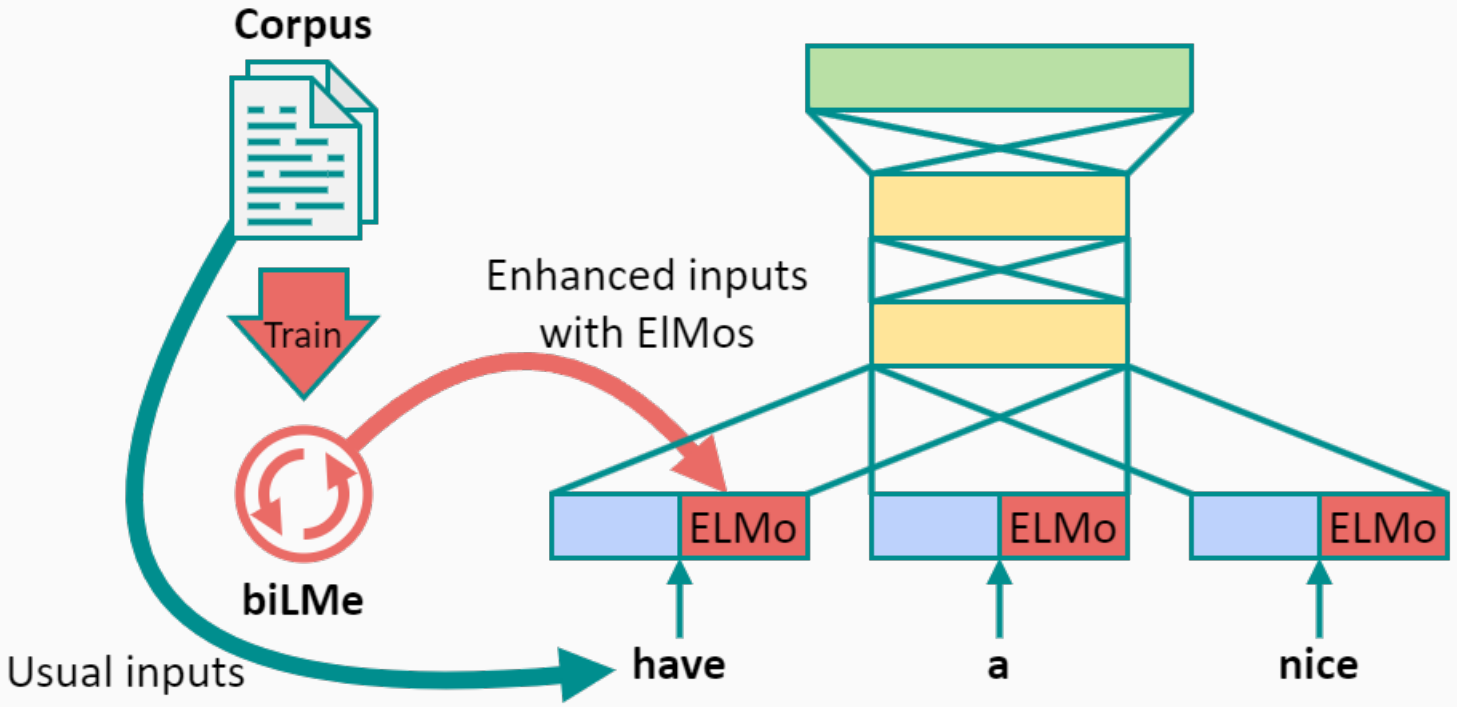
ELMo

- **ELMo** is a deep contextualized word representation that models:
  1. complex characteristics of word use (e.g., syntax and semantics)
  2. how these uses vary across linguistic context (i.e., model polysemy)
- Instead of using a fixed embedding for each word, ELMo looks at the entire sentence before assigning each word in it an embedding
- ELMo combines all the intermediate layer representations from pretrained bidirectional LSTM as the output representation





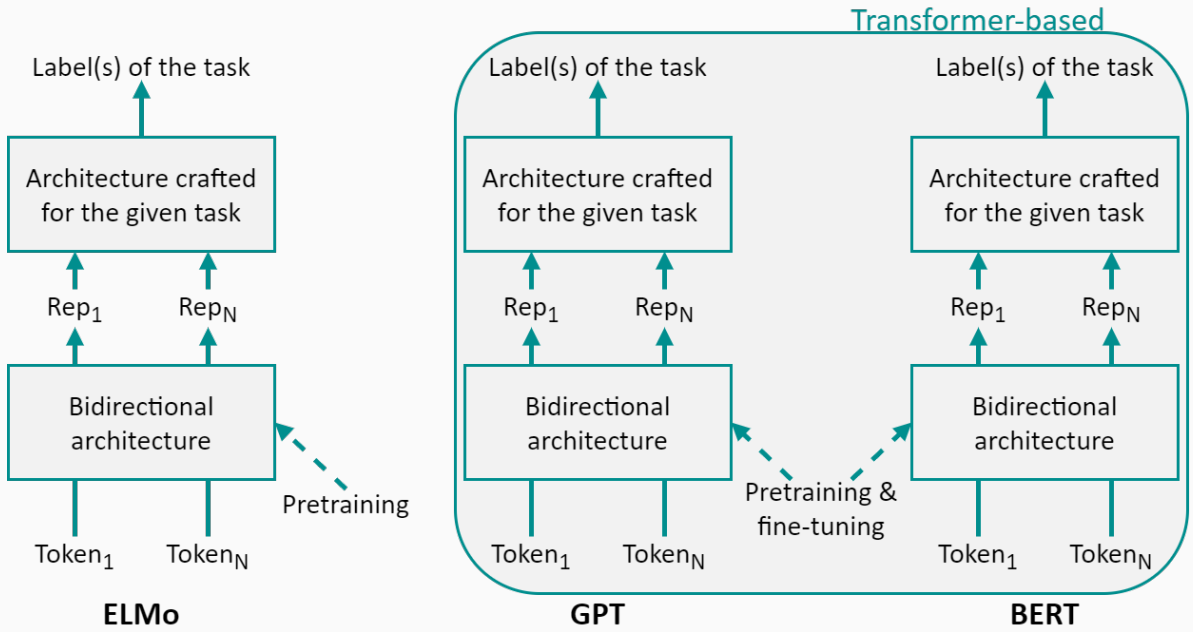
ELMo



ELMo embeddings can be easily added to existing models and significantly improve a broad range of challenging NLP problems, including question answering, textual entailment, and sentiment analysis

From Task-Specific to Task-Agnostic

- Although ELMo has greatly improved solutions to a wide range of natural language processing problems, each solution remains **dependent on a task-specific architecture**
- Transformer-based models (GPT, BERT, etc.) represent an effort in designing general task-agnostic models for context-sensitive representations by fine-tuning all the parameters during supervised learning of the downstream task



Summary

- **Word embedding models** such as Word2vec and GloVe are context-independent. They assign the same pretrained vector to the same word regardless of the context of the word. It is hard for them to handle well polysemy or complex semantics in natural languages
- For **context-sensitive word representations** such as ELMo, representations of words depend on their contexts but they are task-specific
- **Transformer-based word representations** solve this problem by allowing for contextual word embeddings with minimal architecture changes for a wide range of natural language processing tasks

Homework

Lab exercise “Text Vectorization II”

- **Objective:** To provide a basic overview of the following text vectorization techniques:
  - Word2Vec
  - GloVe
  - FastText
  - ELMo

  as well as the usage of the latter to solve a Text Classification task

Bibliography i

Alammar, J. (2019). The illustrated word2vec. *https://jalammar.github.io/illustrated-word2vec/*

Aßenmacher, M. (2020). Modern approaches in natural language processing. *https://slds-lmu.github.io/seminar_nlp_ss20/*

Eisenstein, J. (2018). Natural language processing. *Jacob Eisenstein.*

Godoy, D. V. (2022). *Deep learning with pytorch step-by-step, volume iii: Sequences & nlp*. Independently published.

Jeffrey Pennington, C. D. M., Richard Socher. (2014). Glove: Global vectors for word representation. *https://nlp.stanford.edu/projects/glove/*

Manning, C., Socher, R., Fang, G. G., & Mundra, R. (2017). Cs224n: Natural language processing with deep learning1.

Matthew E. Peters, M. I., Mark Neumann. (2018). A review of deep contextualized word representations. *https://www.slideshare.net/shuntaroy/a-review-of-deep-contextualized-word-representations-peters-2018*

Bibliography ii

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.

Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, *26*.

Pennington, J., Socher, R., & Manning, C. D. (2014). Glove: Global vectors for word representation. *Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)*, 1532–1543.

Rong, X. (2014). Word2vec parameter learning explained. *arXiv preprint arXiv:1411.2738*.

Voita, E. (2020). NLP Course For You. `https://lena-voita.github.io/nlp_course.html`

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2021). Dive into deep learning. *arXiv preprint arXiv:2106.11342*.